Selection Sort

Selection Sort is one of the simplest sorting algorithms and is very useful for introducing the idea of **selecting the minimum (or maximum) element and placing it in the correct position**.

1️⃣ What is Selection Sort?

Selection Sort repeatedly finds the **smallest element** from the unsorted portion of the array and places it at the beginning.

It divides the array into two parts:

- Sorted part (left side)
- Unsorted part (right side)

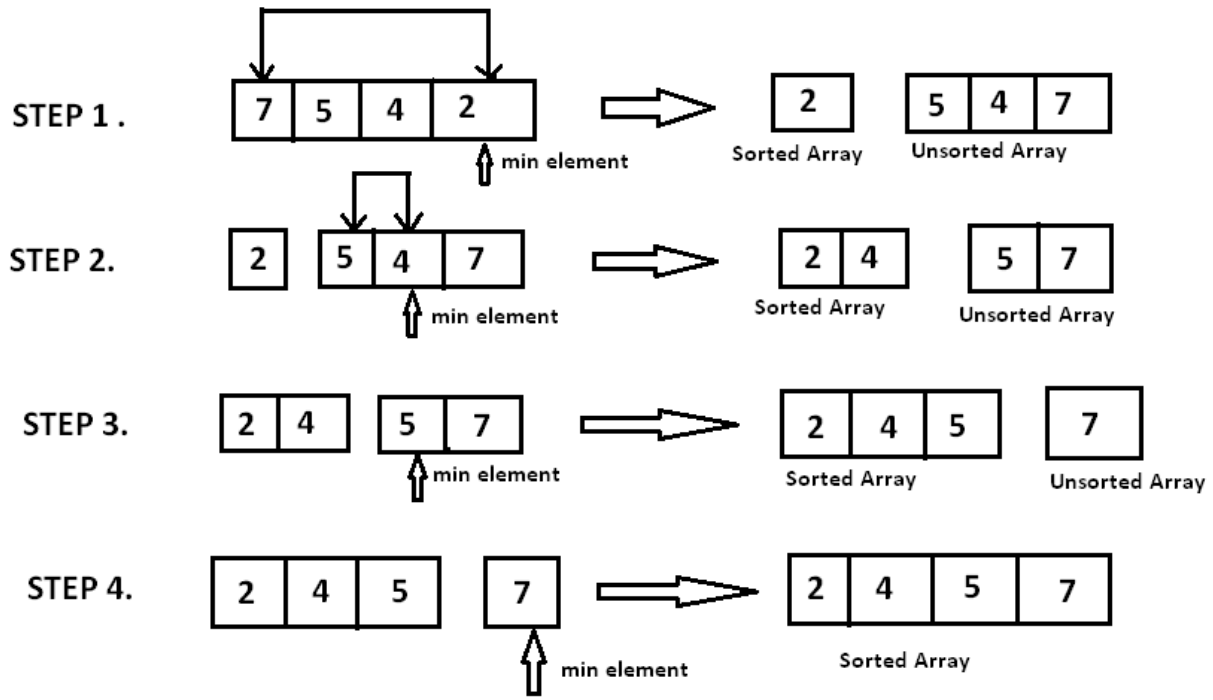
After every pass, one element moves to its correct position.

2️⃣ Basic Idea

For each position `i`:

1. Find the smallest element from index `i` to `n-1`
2. Swap it with element at index `i`
3. Move to next position

3️⃣ Visualization



Example array:

```
64  25  12  22  11
```

Pass 1 → Find minimum (11) → Swap with 64

```
11  25  12  22  64
```

Pass 2 → Find minimum (12)

```
11  12  25  22  64
```

Pass 3 → Find minimum (22)

```
11  12  22  25  64
```

Pass 4 → Find minimum (25)

```
11 12 22 25 64
```

Sorted ✅

4️⃣ Pseudocode

```
SELECTION-SORT(A)
for i = 0 to n-2
    minIndex = i

    for j = i+1 to n-1
        if A[j] < A[minIndex]
            minIndex = j

    swap A[i], A[minIndex]
```

5️⃣ Clean C++ Code

```
#include <iostream>
#include <vector>
using namespace std;

void selectionSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        // Find minimum element
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
```

```
            }
        }

        // Swap with first unsorted element
        swap(arr[i], arr[minIndex]);
    }
}

void printArray(const vector<int>& arr) {
    for (int x : arr)
        cout << x << " ";
    cout << endl;
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};

    cout << "Before Sorting: ";
    printArray(arr);

    selectionSort(arr);

    cout << "After Sorting: ";
    printArray(arr);

    return 0;
}
```

6️⃣ Step-by-Step Example

Array:

```
8 3 5 4
```

Pass 1

Minimum = 3

Swap with 8

```
3  8  5  4
```

Pass 2

Minimum = 4

Swap with 8

```
3  4  5  8
```

Pass 3

Minimum = 5

Swap with 5

```
3  4  5  8
```

Sorted ✅

7️⃣ Time Complexity

Selection Sort always checks all elements.

Best Case

```
O(n²)
```

Average Case

```
O(n²)
```

Worst Case

```
O(n²)
```

Important:

> Selection sort does the same number of comparisons regardless of input.

8️⃣ Space Complexity

```
O(1)
```

It sorts in-place.

9️⃣ Properties

Property	Value
Stable	❌ No
In-place	✅ Yes
Adaptive	❌ No
Swaps	Few
Comparisons	Many

🔟 Why It Is Not Stable?

Example:

```
5a 3 5b
```

After sorting:

```
3 5b 5a
```

Order changed → Not stable.

1️⃣1️⃣ Number of Comparisons

Total comparisons:

```
(n-1) + (n-2) + ... + 1
```

$= \frac{n(n-1)}{2}$

So complexity is:

$O(n^2)$

1️⃣2️⃣ Number of Swaps

Selection Sort makes at most:

```
n-1 swaps
```

Very small compared to Bubble Sort.

1️⃣3️⃣ Comparison with Bubble Sort

Feature	Bubble	Selection
Swaps	Many	Few
Comparisons	Many	Many
Stable	Yes	No
Best Case	O(n)	$O(n^2)$

Feature	Bubble	Selection
Adaptive	Yes	No

1️⃣4️⃣ Comparison with Insertion Sort

Feature	Selection	Insertion
Best Case	$O(n^2)$	$O(n)$
Adaptive	No	Yes
Stable	No	Yes
Speed	Slow	Faster
Swaps	Few	More shifts

1️⃣5️⃣ When To Use Selection Sort

Good for:

- Small datasets
- When swaps are expensive

Not good for:

- Large datasets

1️⃣6️⃣ Loop Invariant (DAA Theory)

At start of each iteration:

```
A[0 ... i-1] is sorted
```

After iteration:

```
A[0 ... i] is sorted
```

1️⃣7️⃣ Viva Questions

1. Why is selection sort $O(n^2)$?
2. Why is it not stable?
3. How many swaps occur?
4. Is selection sort adaptive?
5. Compare with insertion sort.

1️⃣8️⃣ Short Exam Answer

Selection Sort repeatedly selects the smallest element from unsorted portion and places it at the beginning.

Time complexity: $O(n^2)$.

Space complexity: $O(1)$.

Not stable and not adaptive.